1 Classification, Regression and Clustering

These answers will deal with a single dataset, browser analytics events with backing system RPCs. The business has 10m users. ARPU is currently USD \$160/$user/year$ [Stand]. Total revenue is USD \$1.6billion. Each session will generate 100 clicks, and 100 front end RPCs. Each front-end RPC generates 100 back end calls. Users will interact with the system daily. To deal with seasonal variations in data, we need to store and process at least a full year's worth of data.

Therefore the **volume** and **velocity** of data involved is roughly:

variable	**description**	**variable**	**description**
S_d	sessions per day = 1	C_s	clicks per session = 100
F_s	front-end RPCs per session = 100	B_f	back-end RPCs per F_s = 100
E_s	events per session	B_e	bytes per event = 100
B_s	bytes per session = 1000	T_b	total data size
N_d	total number of days stored	T_e	total events stored analyzed per user

$$
\begin{aligned}
E_s &= C_s + F_s + F_s * B_f \\
T_e &= N_d * S_d * E_s \\
&= N_d * S_d * (C_s + F_s + F_s * B_f) \\
T_b &= (S_d * N_d * B_s + T_e * B_e) * U \\
&= 3.73 \text{ petabytes per year} \\
&= 946 Mb/s.
\end{aligned}
$$

big data classification example

A model can be trained to predict if a user may be a fraudster, using the user event stream, including a "deemed fraudster" event. The "deemed fraudster" event would be removed from the stream and used as the label for training.

Since the dataset is labelled, this is a **classification** problem.

If the company cannot keep payment fraud under control, processors will increase merchant fees, or even disconnect the company from payment networks. The fraud loss ratio is currently 1.8% of revenue [Dua21], providing a **value** of USD \$32.4million.

Both fraudsters and the system will change. The model will need to be retrained frequently. If the model is retrained every month, and the data should be no more than 2 weeks old at launch, the system needs to process $\frac{3.73PB}{2\ weeks} = 25Gbps$.

big data regression example

The dataset includes a "purchase" event, with the cart value. We can label the user with the total spent over the past year. The goal is to predict a customer's total spend from the event stream. Since the dataset is labelled with a continuous value, it is a **regression** problem.

The ability to better target spending is of significant **value** to the business. A 1% improvement in transaction completion is worth USD $18million

big data clustering example

The model should detect significant differences in sessions - too short, too long, or containing strange interactions.

There aren't any labels - this is a **clustering problem**. Every completed session's event stream will be evaluated against the model to determine if it is an anomaly, where "anomaly" is defined as "being too far away from any of the clusters".

The alerting system would count sessions reported anomalous by the model, and if they exceed a certain rate (or ratio), trigger an alert. The business **value** is in reducing revenue loss and escalations. Going from 3 9's to 5 9's of downtime would be worth USD $2million in immediate revenue.

In Anomaly detection, we again want short training intervals, with a lag in the training data window to avoid boiling the frog as bad features are incrementally rolled out. This would still require training every month, requiring $25Gbps$ of processing.

2 Feature Selection/Construction Methods

2.1 feature ranking/feature weighting vs feature subset selection methods

Feature Ranking is a global ordering of features, where each feature is evaluated once. This will then be used by a greedy subset search algorithm which will pick the top-K features [Ngu26b].

Feature Subset Selection will iterate over subsets and evaluate each subset, selecting the best from those reachable using the algorithm.

Feature Ranking is very quick to perform typically $O(N)$, but does not consider interactions between features. Feature Subset Selection can take significantly more time, a complete search would require 2^N iterations.

Pearson Correlation Coefficient [Pea95], measures the linear relationship between a feature and a class label.

Information Gain, [Qui86] is derived from the entropy/purity/ratio of classes before and after a {feature, value} pair splits the dataset.

Feature Subset Selection is a search algorithm, producing candidate subsets for evaluation. Sequential Forward Search [Ngu26a] grows the proposed feature set by adding the "best" feature from the remaining set. The algorithm completes when the set exceeds a preconfigured threshold.

Beam Search [Low76], grows N sets. At each iteration, the N sets are grown, with the best N results kept. This allows the search to spread out and test hypotheses.

2.2 Define Filter, Wrapper, Embedded Feature Selection/Construction

Filter feature selection/construction methods use a subset search without training a ML model.

Primary Component Analysis [FRS01] translates the features to a new set of orthogonal features using Eigenvectors. The features are sorted, and the top-K are selected.

Relief [KR92] randomly picks values from the dataset, using Euclidean distance to find features that are able to find class boundaries.

Wrapper feature selection methods contain a subset selection algorithm and the target machine learning algorithm is used to evaluate subset quality.

Sequential Backwards Search [Ngu26a] uses the same approach as Sequential Forward Search, beginning from a full set, and removing the "worst" feature at each stage. Yu, L [LY05] recommends using set-cover to choose between forwards or backwards searches.

Sequential Floating Forward Selection SFFS[PNK94] alternates between adding and removing features from the set. This algorithm is not guaranteed to converge, so any implementations will need to keep track of already visited sets.

Embedded feature set selection/construction algorithms derive the feature subset during construction of the model.

CART DecisionTree[Bre+17] can be used for both feature selection and feature construction. The feature list from the nodes of the tree is the feature

subset. The boolean equations in the nodes can be considered new boolean features, allowing rows to be mapped onto the path through the tree.

Support Vector Machines [CV95] finds the slope of a line between two classes that maximizes the distance between values in the class by mapping into a new set of dimensions.

2.3 Compare and contrast the performance of supervised filter, wrapper, and embedded methods

Filter methods generalize best to different problems, are generally cheap and provide the worst accuracy. Filter methods are cheap because they use cheap evaluation functions (linear). Filter methods do not evaluate the subsets in the target model, limiting the resulting accuracy. The subset's performance will be similar between models, allowing it to generalize better.

Wrapper methods generalize the worst, are very expensive but provide the best accuracy. Wrapper methods train an instance of the target model at least once per subset tested. Testing against the target model with subset search is more likely to find subsets containing features that work well together, but have limited individual predictive power. This gives wrapper methods high accuracy, but low generality. The feature subset is linked to the target model algorithm.

Embedded methods are in the middle, providing accuracy, cost and generality between the other two. Embedded methods have the same cost as Filter (both train the target model once), and are cheaper than Wrapper methods. Since Embedded only trains a single model, it will be cheaper than a Wrapper method which trains N models for each subset. Embedded methods perform feature selection during model construction.

2.4 Correlation-based Feature Selection (CFS)1

Yu, L. [LY05] states that CFS is a Filter method. Reading Hall M.'s paper [Hal00], CFS doesn't use a model, but does validate subsets instead of features individually. Since it doesn't use a model, it can't be embeded or wrapper, leaving filter the remaining class. CFS uses the BestMatch complete subset search algorithm, although Hall recommends adding an early exit criteria. Any complete search will produce similar results. Single forward search was also tested, but did not perform as well.

3 Use KNIME to perform Feature Selection

3.1 (i) Naive Bayes

KNime image: NB pre-split dataset

Data Set	Accuracy
Training	82.86%
Test	80.19%

Table 1: 3.i.a naive bayes, pre-split dataset.

KNime image: NB split with fixed seed dataset

Data Set	Accuracy
Training	86.21%
Test	85.85%

Table 2: 3.i.b naive bayes, split dataset.

The two datasets produced different training and test results. This is expected since the data is randomly assigned to the training and test sets. To obtain a more accurate stable measure of the accuracy of the model, K-fold cross validation should be performed on the training data. That will give an average and std deviation of the model, allowing the model's stability to be assessed.

3.2 (ii) Forward Feature Selection

KNime image: FSF on full dataset
Features Selected: F1-5, F7, F9, F11, F13, F15, F22-28
By performing feature selection without splitting the data, the results embed bias. We won't know if the model generalizes well or if it is overfitted to the dataset.

3.3 (iii) Forward Feature Selection

KNime image: NB with unsplit feature selection dataset

Data Set	Accuracy
Training	91.02%
Test	89.62%

Table 3: 3.iii improperly filtered dataset.

The increase in training accuracy shows that feature selection is improving the accuracy. However, the testing results cannot be trusted to demonstrate generalization. Dropping unimportant features should allow the remaining features to provide a stronger signal.

3.4 (iv) Forward Feature Selection - correct

Note: Test set filtering is not required, the model ignores unknown features.

KNime image: NB with properly split feature selection dataset

Data Set	Accuracy
Training	91.43%
Test	89.62%

Table 4: 3.iv correctly filtered dataset.

Features Selected: F3,F4,F7,F23

Comparing with i(a), we see a 9% improvement in training and test accuracy. Naive Bayes assumes that features are independent. If the features are not independent, the model will be more confident - there will be mulitple sources of confidence for the same answer. Feature selection will result in a better performing subset, increasing feature independence.

We don't see significant changes in accuracy from the improperly done feature selection. However, the split did give us a much smaller feature set and we see the model generalizes.

3.5 (v) C4.5 Decision Tree

Note: Decision Tree View (simple) does not appear to be available, I converted the tree to an image instead.

KNime image: C4.5 Decision Tree

Data Set	Accuracy
Training	97.55%
Test	91.51%

Table 5: 3.v C4.5 Decision Tree.

KNime image: C4.5 Decision Tree Structure

At each point, the algorithm picks the best split point. The feature subset can be extracted from the tree by combining the features from all the nodes.

3.6 (vi) PCA with C4.5 Decision Tree

KNime image: PCA Filtered C4.5 Decision Tree

Data Set	Accuracy
Training	95.92%
Test	88.68%

Table 6: 3.vi PCA filtered C4.5 Decision Tree.

See section 4 to see the top-5 principal components. The top-5 eigenvalues were:

8.712990413236952, 3.9565684550423805, 2.752964596790358, 2.4668984522889423, 1.980269212245705

While the training results were better, the test accuracy was worse. I feel this means there are non-linear relationships the regular tree is able to find. PCA would lose that information during the transformation.

3.7 (vii) Correlation-based Feature Selection

Note: Feature1 dropped due to no variance

KNime image: CFS

Data Set	Accuracy
Training	88.16%
Test	87.74%

Table 7: 3.vii CFS filtered Naive Bayes.

Features Selected: F2-3, F5-9, F12-15, F20, F24-25, F27-29, F30 Wrapper Features Selected: F3,F4,F7,F23

Naive Bayes using the wrapper produced a test accuracy of 89.62%, while CFS returned 87.46%. The wrapper algorithm trained 561 models, while CFS had 528 iterations (FeatureSelectionLoopStart.maxIterations). The algorithm is hard to implement in CNIME, making the execution of CFS take approximately twice as long. Interestingly, the two feature sets only share F3, and the wrapper method produced a much smaller feature set. The wrapper algorithm is better able to reduce the actual error in the resulting model at each step.

References

[Pea95] Karl Pearson. "Note on regression and inheritance in the case of two parents". In: *Proceedings of the Royal Society of London* 58 (1895), pp. 240–242.

[FRS01] Karl Pearson F.R.S. "LIII. On lines and planes of closest fit to systems of points in space". In: *Philosophical Magazine Series 1* 2 (1901), pp. 559–572. URL: https://api.semanticscholar.org/CorpusID:125037489.

[Low76] Bruce T. Lowerre. "The Harpy Speech Recognition System". 1976.

[Qui86] J. R. Quinlan. "Induction of Decision Trees". In: *Machine Learning* 1 (1986), pp. 81–106.

[KR92] Kenji Kira and Larry A. Rendell. "The Feature Selection Problem: Traditional Methods and a New Algorithm". In: *AAAI Conference on Artificial Intelligence*. 1992. URL: https://api.semanticscholar.org/CorpusID:46457448.

[PNK94] P. Pudil, J. Novovičová, and J. Kittler. "Floating search methods in feature selection". In: *Pattern Recognition Letters* 15.11 (1994), pp. 1119–1125. ISSN: 0167-8655. DOI: https://doi.org/10.1016/0167-8655(94)90127-9. URL: https://www.sciencedirect.com/science/article/pii/0167865594901279.

[CV95] Corinna Cortes and Vladimir Naumovich Vapnik. "Support-vector networks". In: *Machine Learning* 20 (1995), pp. 273–297.

[Hal00] Mark Hall. "Correlation-based Feature Selection for Discrete and Numeric Class Machine Learning". In: Jan. 2000, pp. 359–366.

[LY05] Huan Liu and Lei Yu. "Yu, L.: Toward Integrating Feature Selection Algorithm for Classification and Clustering. IEEE Transaction on Knowledge and Data Engineering 17(4), 491-502". In: *IEEE Transactions on Knowledge and Data Engineering - TKDE* 17 (Apr. 2005), pp. 491–502. DOI: 10.1109/TKDE.2005.66.

[Bre+17] Leo Breiman et al. *Classification and regression trees*. Routledge, 2017.

[Dua21] 2021 Dual Payments. *Chargeback Statistics*. [Online; accessed 9-March-2026]. 2021.

[Ngu26a] B.H. Nguyen. *AIML427 Feature Manipulation*. 2026. URL: https://ecs.wgtn.ac.nz/foswiki/pub/Courses/AIML427_2026T1/LectureSchedule/Week2_FeatureManipulation.pdf.

[Ngu26b] B.H. Nguyen. *AIML427 Filter Feature Selection*. 2026. URL: https://ecs.wgtn.ac.nz/foswiki/pub/Courses/AIML427_2026T1/LectureSchedule/Week3_FS_Filter.pdf.

[Stand] n.d. Statista.com. *Facebook average revenue per user (ARPU) as of 4th quarter 2023, by region*. [Online; accessed 9-March-2026]. n.d.

4 Appendix - Features

Eigenvalue: 8.712990413236952
FeatureVector: 0,0.11199022986101669,-0.10303021919094062,0.16815076555816535,-0.1060047154229859,0.20791921154974005,-0.16152211161768934,0.23480440275633543,-0.15953740406422262,0.24872695636743872,-0.1337943469074957,0.2794105900437599,-0.07512049777150774,0.2894785629780161,-0.10876769831122128,0.27570165755626874,0.00179238545(
0.03608722053709462
Eigenvalue: 3.9565684550423805

FeatureVector: 0,-0.10623526016439234,-0.10533253814837638,-0.12655469049655385,0.03818023881
0.136868138990739,-0.08104041299172997,-0.09213411640470774,-0.07765434316750879,-
0.09825032794826052,-0.16635024325027592,-0.13197251978769473,-0.23200776171152912,-
0.11329732256313002,-0.24045123062447926,-0.026799059112381205,-0.26697535283159807,-
0.00309924243882642,-0.39561512287516853,0.04957616236190548,-0.33008875014801625,0.0544612413
0.28397896155427915,0.12747534738441746,-0.2615397290475803,0.11866913443193763,-
0.27486075264175835,0.15253184314915547,-0.2433247149864297,0.15714404827358391,-
0.10143388600997148,0.12274677528655142,-0.10370669770796156 **Eigenvalue:**
2.752964596790358
FeatureVector: 0,0.32439304827175375,0.192713595706209,0.25945585400047,0.23352072918850086,0
0.08814007481323394,0.30146837754966216,-0.08179084542707214,0.23455228538544293,-
0.05836976145352405,0.12693376330336056,-0.12445570319343335,-0.031979592214529055,-
0.059270764428783165,-0.04897568751795348,0.021731225619893508,-0.15217748696155603,0.06468227
0.1434713087258693,0.1964300978331025,-0.1782681050569509,0.04539407515486823,0.05690120951076
Eigenvalue: 2.4668984522889423
FeatureVector: 0,0.17538598546231507,0.05166655583172295,0.15659791914802845,-
0.14679828407404175,0.17681796868609403,-0.07509659698295458,0.19311845805095695,0.08451164798
0.008461142841062143,0.0064367819923778725,-0.057243746899859935,0.04555080152536112,-
0.0675534348524617,0.02378248720144592,-0.12157990609777602,0.052783351728858605,-
0.13352056808300186,-0.23608618422908664,-0.24526744771577808,-0.2918779888224897,-
0.10204165254307083,-0.2635122951536268,0.023713859299413984,-0.4907918946447774,0.06115510200
0.42608742827600476
Eigenvalue: 1.980269212245705
FeatureVector: 0,-0.04706341589402093,0.468805832707772,0.002504656308596978,-
0.42783822765145557,-0.10757608349258725,0.03257118337644784,-0.1199964152529897,0.00745003130
0.035545886122166755,0.025043679768620774,-0.058678390826969254,0.004462911783591111,-
0.36837662075472444,-0.029372574901969335,-0.045356403021704024,-0.062071377282175676,-
0.007183575567238008,0.1374546037484637,0.3098276843476896,0.06793449022629393,-
0.23666394292691045,0.22494513177587194,0.032170854466240184,0.0513136162879607,0.37805285693
0.01911405995229084,-0.15910786485313774,-0.08787117918713866,-0.0064792718164519885

5 Appendix - Figures

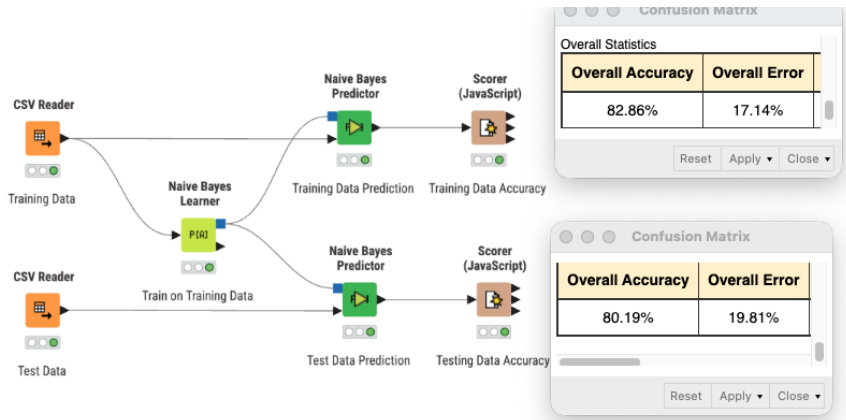

Figure 1: NB pre-split dataset

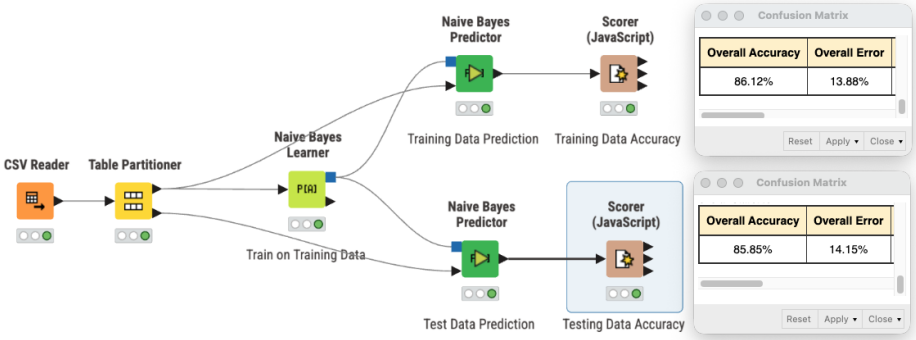

Figure 2: NB split with fixed seed dataset

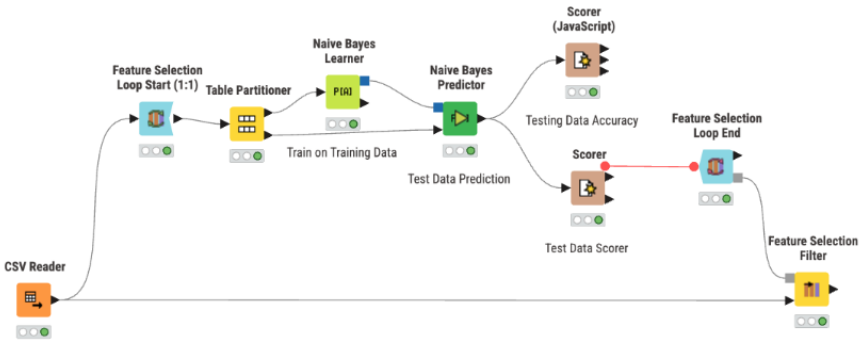

Figure 3: FSF on full dataset

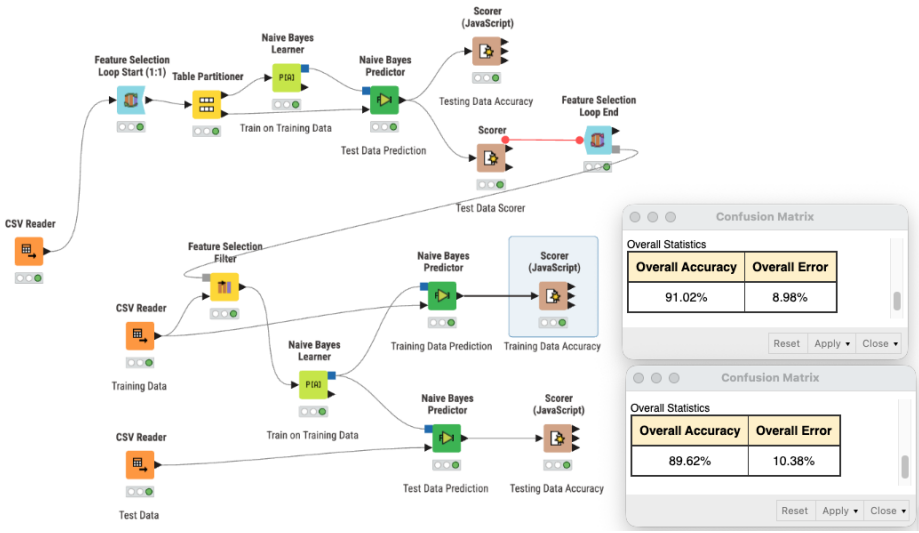

Figure 4: NB with unsplit feature selection dataset

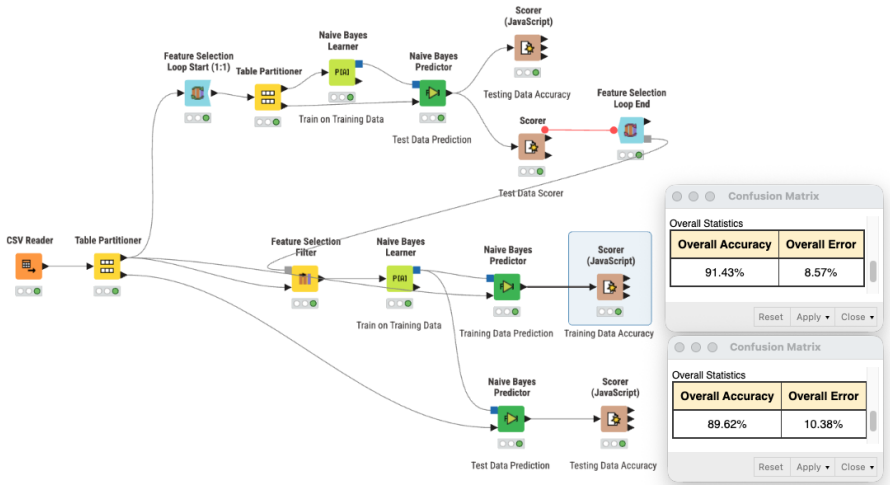

Figure 5: NB with properly split feature selection dataset

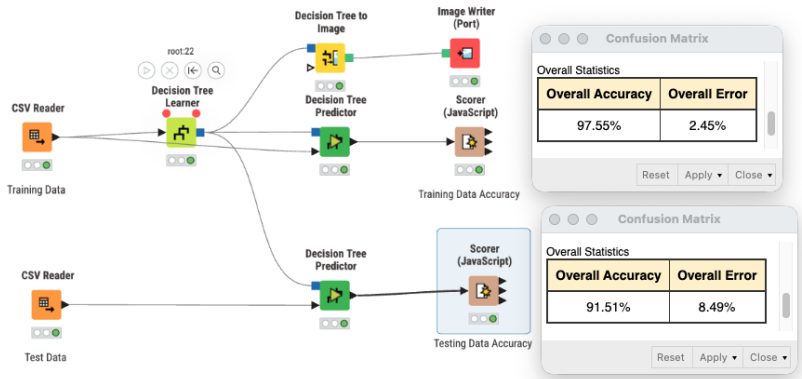

Figure 6: C4.5 Decision Tree

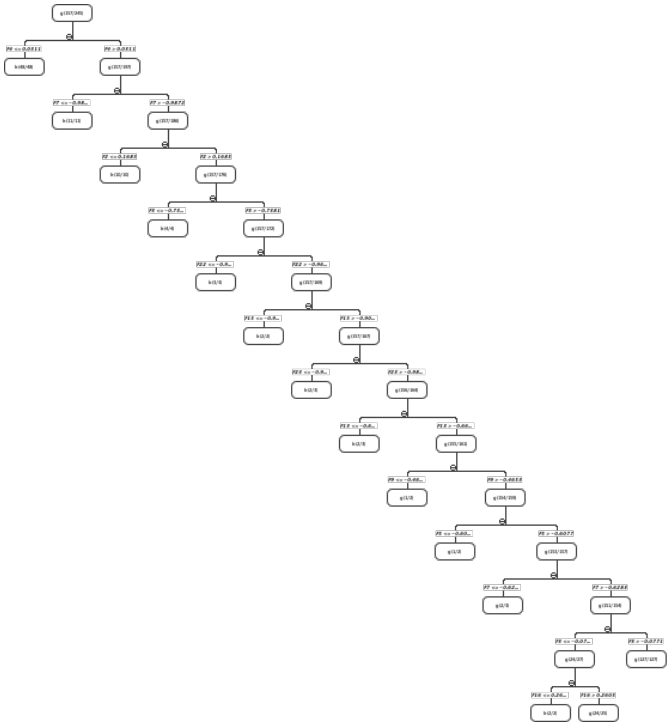

Figure 7: C4.5 Decision Tree Structure

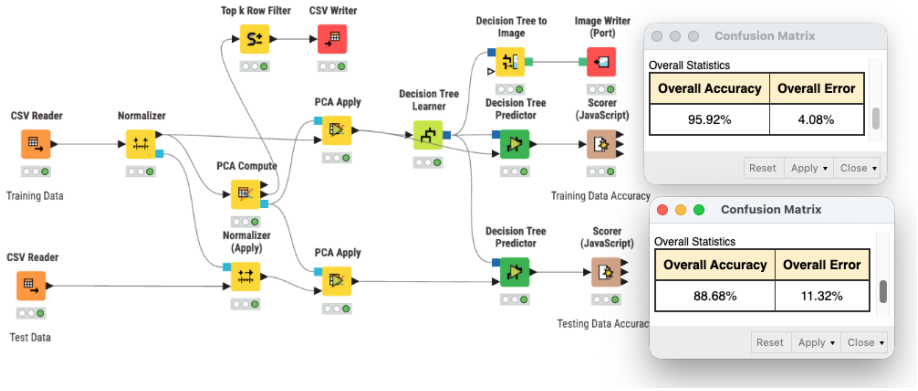

Figure 8: PCA Filtered C4.5 Decision Tree

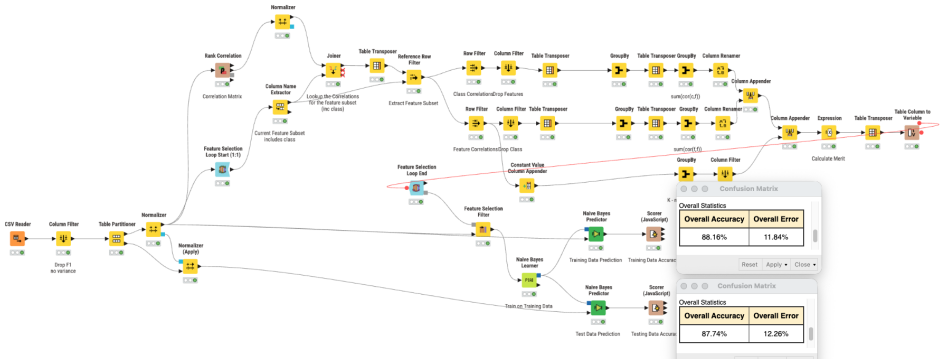
Normalizer
Rank Correlation
Correlation Matrix
Joiner
Table Transposer
Reference Row Filter
Lookup the Correlations for the feature subset (inc class)
Extract Feature Subset
Column Name Extractor
Current Feature Subset includes class
Feature Selection Loop Start (1:1)
Row Filter
Column Filter
Table Transposer
Class Correlations
Drop Features
Feature Correlations
Drop Class
Constant Value Column Appender
Feature Selection Loop End
Feature Selection Filter
GroupBy
Table Transposer
GroupBy
Column Renamer
sum(corr(f,c))
Column Appender
Expression
Calculate Merit
Table Transposer
Table Column to Variable
CSV Reader
Column Filter
Drop f1 no variance
Table Partitioner
Normalizer
Normalizer (Apply)
Naive Bayes Learner
Train on Training Data
Naive Bayes Predictor
Training Data Prediction
Scorer (JavaScript)
Training Data Accuracy
Test Data Prediction
Testing Data Accuracy
Confusion Matrix
Overall Statistics
Overall Accuracy
Overall Error
88.16%
11.84%
Reset
Apply
Close
Confusion Matrix
Overall Statistics
Overall Accuracy
Overall Error
87.74%
12.26%

Figure 9: CFS